

Bitcoin Price Prediction Project Report

By Anoop Bhaskar (abhas2) and Subin Pradeep (subinpp2)

1. Introduction

Bitcoin, a well-known cryptocurrency, has revolutionized the financial markets industry and changed the way payments are made in the 21st century. Bitcoin is known for its extremely volatile and high volume market natures, making it a complex yet interesting stock to analyze for statistical prediction methods.

This project focuses on predicting Bitcoin's market price using the machine learning and model selection techniques we have learned so far in this class. For the sake of this project, we are using a dataset inspired by Kaggle that combines blockchain metrics, mining statistics, and transaction activity to capture different aspects of the Bitcoin network. To prepare the data for modeling, we analyze outliers, correlations, and multicollinearity to reduce redundancy and improve feature quality.

The primary objective is to refactor the dataset into a form suitable for predictive modeling, apply statistical learning methods including Multiple Linear Regression (MLR), Ridge Regression, and Lasso Regression, and evaluate their performance across different forecasting horizons (e.g., one day ahead, three days ahead, etc.).

Through this process, the project aims to explore how different regularization techniques handle collinearity, how predictive accuracy degrades over longer horizons, and what insights can be drawn about predicting a stock as volatile as Bitcoin.

2. The Data Cleaning Process

The original Bitcoin dataset contains a plethora of features and provides valuable time-series data between the years 2010 and 2018 for our prediction task. While this dataset is rich in its content, there are numerous challenges that needed to be addressed before the dataset was fit for model training.

1. Initial Data Cleaning

To begin our data cleaning process, we examined the data types of the columns in our dataset and found that all the features are numerical. Since there were no categorical data types within our dataset, there was no need to refactor/transform any of the columns. For the next step of our initial data cleaning process, we decided to trim our dataset of the values in the dataset where market price was equal to zero. When examining the dataset, we noticed that the initial time-series data, when bitcoin was first listed on the market, contained many entries where market price was zero. For the purpose of our model, we decided to trim out these entries and start the dataset at the first entry where market price was a non-zero value.

2. Handling Missing Values

To identify missing values in our dataset, we calculated the null values per feature. From these calculations, we identified that there were 21 null values for the `btc_trade_volume` variable. To properly handle these missing values, we looked through many different techniques. Given that we are working with time-series financial data, we decided that it would be best to do some sort of imputation for the missing values. We settled on using spline interpolation for imputation. Spline interpolation fills in the blanks in our data by drawing a smooth, flexible curve through the existing non-null data points. It identifies neighboring data points before and after the null value, then fitting a polynomial spline curve to pass through these points. It ensures there is a measure of smoothness in the missing data. More specifically, it works well with time-series financial data because it generates a continuous curve that realistically models the nature of price and rate movements between data points.

3. Outlier Detection

Detecting and handling outliers for this dataset was a bit tricky given the context of our dataset. Bitcoin was an asset that moved from a niche technology valued at single digits to a global asset worth thousands of dollars. In the data, it is visible that there was a massive spike in 2017 where the price rocketed garnering more media coverage and investment opportunities. The problem might be more apparent now: How can we determine what data points are outliers if we have so much varying data throughout the years? Furthermore, should we remove these outliers from our data given that many of them serve as historical data points? The nature of our dataset is important to note. Financial outliers are important as they can capture crashes, policy changes, or market shocks. Therefore, instead of

removing them, we decided to flag each data point that was considered an outlier. We calculated outliers in each column by analyzing the rolling window mean for a window of 30 days. Since there are 20+ features, we decided to create a new column called `'is_any_outlier'`. The `"is_any_outlier"` variable is just a summation style column, meaning if a data point was considered an outlier by any column it would have a 1 in its place and 0 otherwise. This was done as it could be useful for future analysis.

4. Data Transformation

Looking at our variables, we realized that many of them are skewed. To address this, we performed a log transformation on all the variables which showed skewed distribution. This includes the target variable (market price). This was done as financial time series data is multiplicative, meaning that a 10% change is more meaningful than the price of something changing by \$500 in either direction. Log transformation makes these % changes more linear and easier for models to capture. Additionally, this is pretty common in financial forecasting. In turn, this means that our model will be predicting $\log(\text{'btc_market_price'})$.

5. Standardizing Data

Once we transformed our data to reduce skewness, we were able to standardize our data. This is an essential step of our dataset preparation as it ensures fair comparison between all our features. Without standardization, the features with a greater spread of values dominate the features with less variability. We standardized our data using the `StandardScaler` function in the `sklearn` package.

6. Multicollinearity

We also analyzed the correlation between our features and target variable to prevent any sort of multicollinearity within our dataset. To assess the multicollinearity in our dataset, we created a heatmap displaying the correlation between all variables within the data. Looking at this heatmap, we discovered that there were many features displaying exceptionally high correlation numbers indicating collinearity. To address this issue, we calculated the VIF scores for our features and removed variables from our working dataset that had a VIF score that was considered "significantly" high or "significantly" low. After this filtering process, we were left with the following features for our final dataset for prediction: `"btc_n_unique_addresses"`, `"btc_n_transactions_excluding_chains_longer_than_100"`, `"btc_estimated_transaction_volume_usd"`, `"btc_avg_block_size"`, `"btc_n_transactions_per_block"`, `"btc_total_bitcoi`

ns","btc_trade_volume","btc_transaction_fees","btc_hash_rate","btc_cost_per_transaction","btc_median_confirmation_time", and "is_any_outlier". Within these features, there were still a few variables with VIF scores greater than 10 indicting that there may still be some multicollinearity within these final features. However, we decided to keep them and utilize Ridge and Lasso regression methods in the model building stage of our project to deal with any potential leftover multicollinearity.

7. Lags and Rolling Windows

Now that our data was cleaned, we moved onto the final step of our data preparation which was to create lags and rolling windows for our features. In time series modeling, past observations often carry predictive power for future outcomes. Bitcoin's market price exhibits temporal dependencies, where both short-term fluctuations and medium-term momentum can influence subsequent values. To capture these dynamics, lagged features were engineered for the target and finalized predictors. By including these lags, the models were able to "look back" across multiple temporal horizons, improving their ability to learn both short-lived and recurring structures in the data. Importantly, lagging ensures strict causality: only past information is used to predict future prices, eliminating the risk of information leakage.

While lagged features provide our model with discrete past points, they do not reflect any meaningful information regarding trends. To address this, we created rolling window statistics for each feature in the form of seven-day rolling averages and seven-day rolling standard deviation values. We deliberately chose a 7-day window as this most aligns with weekly market cycles, during which both retail and institutional trading behavior often resets. Furthermore, a one-week horizon balances smoothness with responsiveness. Longer windows would obscure short-term dynamics, while shorter ones would fail to filter noise effectively. Together, the lagged and rolling window features enhanced the dataset with both temporal depth and trend context. While lagged variables represent discrete historical anchors, rolling features summarize the recent environment in which Bitcoin's price evolved. This dual representation will enable our models to detect persistent effects, medium-term momentum, and volatility clustering.

8. Train/Test Split

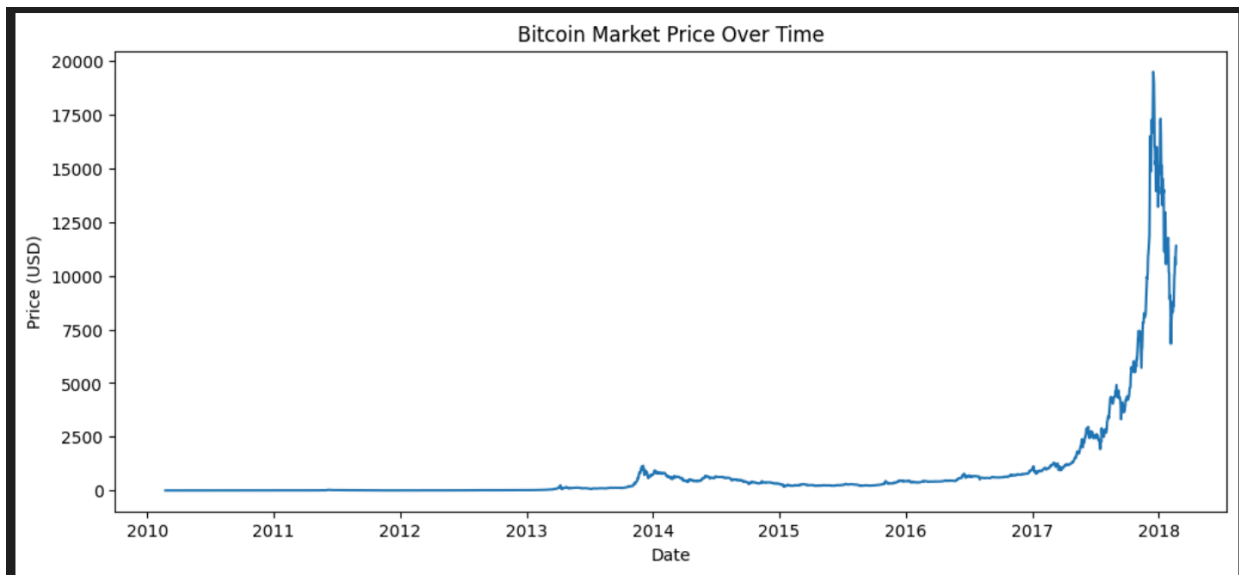
Finally, our data was ready for train/test splitting. Since we are working with time-series data, we cannot implement a random split within our dataset. Therefore, we decided to just split the data with a 80/20 train/test split sequentially. The first 80% of the dataset was used for training while the final 20% of the dataset was used for testing. Essentially, data from 2010 until mid 2016 was

used as training data, and the remaining data from mid 2016 until 2018 was used as testing data.

3. Methodology and Model Selection

Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the underlying patterns, relationships, and distributions within the reconstructed dataset. From our data analysis, the most important take away we found from the dataset was from the following plot of Bitcoin market price vs data:



From the above plot, we quickly notice the volatility and rapid growth that Bitcoin encountered within the given time frame. It is important to note that Bitcoin has two "eras" we can define using the above plot. There is an "era" of initial release which includes the early days of Bitcoin between the years 2010 and 2016. In this "era" there is very limited growth and movement in the market price which makes sense as this was the time when Bitcoin was first released and didn't have much traction. The next "era" we observe includes the days of Bitcoin between 2017 and 2018. During this "era," Bitcoin is extremely volatile and takes major jumps indicating that during this time Bitcoin gained popularity and traction at a rapid rate.

These "eras" are important to note as it presents our model with the challenge of predicting data from the "explosive era" while being trained on data from the "initial release era." At first thought, this seems difficult but we will examine the model's

performance and assess how well it does in predicting market price despite these challenges.

Framework

The goal of this project was to predict Bitcoin's daily market price using blockchain and market-derived predictors. Since the dataset is longitudinal, conventional random train-test splits would introduce lookahead bias. To preserve the temporal ordering of data, a time-series split was implemented: the first portion of the data used for training, and the later portion reserved for testing. This design ensures that the models are always trained on past observations and evaluated on future ones, mirroring real-world forecasting conditions.

Model Development

1. Baseline Approach (Multiple Linear Regression)

The first model applied was Multiple Linear Regression (MLR). The primary purpose of using MLR was selected for its interpretability and to establish a baseline against which the benefits of regularization could be compared. Coefficient estimates were inspected to understand directional relationships between predictors and the target. As a starting point, Multiple Linear Regression was applied on the engineered features dataset. An important idea to note here is that our engineered features did indicate some leftover multicollinearity. This is important because MLR can suffer from multicollinearity where many predictors are correlated. It can lead to inflated variance in coefficients and unstable predictions. Even though we knew that MLR would properly perform the worst out of our model selections, we decided to still implement it in order to observe the results and verify that it really does struggle on data that contains collinearity.

2. Regularized Regression Approaches

To address collinearity issues within our data, we decided to implement Ridge and Lasso Regression methods.

Ridge Regression works by shrinking all coefficients toward zero but does not perform feature elimination. Based on theory, it is well-suited for multicollinear data, since it distributes weights across correlated features and reduces variance making it an ideal candidate for our dataset.

Lasso Regression works by shrinking coefficients and enforces sparsity by negating some to exactly zero. This makes it valuable in high-dimensional settings, such as the engineered Bitcoin dataset with numerous lag and rolling features. Furthermore, our analysis indicated that Lasso effectively reduced the feature

space, often retaining a subset of variables most correlated with near-term price changes.

Using a combination of both Ridge and Lasso Regression methods on our dataset provides us with two regularization methods we can use to compare against the baseline MLR model.

Horizon Forecasting

To evaluate our models according to project guidelines, the models were tested across multiple forecasting horizons: t , $t+1$, $t+3$, $t+7$ where t is a given day. For each horizon, features up to day t were used to predict the price at $t+h$. The purpose of this approach was to compare our models benchmarks in predicting current vs future day's market price given historical data. We chose to examine a maximum of 7 days ahead as it made the most sense given the nature of market trends. We figured that a model's accuracy on larger time frames (2 weeks, 1 month, etc.) would only yield lower results.

Model Evaluation

Before training the models, several hypotheses guided the choice of our evaluation metrics.

For our Multiple Linear Regression model, we hypothesized that it would provide a strong baseline benchmark but would most certainly struggle with the collinearity present in our dataset leading to unstable coefficients and potentially inflated variance in its predictions. We still expected this model to perform decently well given we removed almost all of the collinearity within the dataset during the cleaning process. For Ridge Regression, we hypothesized that it would outperform MLR because of its ability to stabilize coefficient estimates under multicollinearity. For our Lasso Regression, we hypothesized that it would excel given the large number of correlated and engineered predictors like the lags and rolling windows features we created. We expected this model to perform the best out of the three given that it could reduce noise and isolate the most relevant signals for our prediction task.

Given these hypotheses, the following metrics were employed:

Root Mean Squared Error (RMSE): We chose this metric because large forecasting errors are especially costly in financial contexts.

Mean Absolute Error (MAE): We selected this metric to provide a more interpretable measure of average daily deviation, less sensitive to extreme outliers than RMSE.

R^2 (Coefficient of Determination): Finally, we chose this metric to measure how much variance in Bitcoin's price is explained by the models.

Together, these metrics were intended not just to rank models, but to validate or reject the hypotheses about how each method would handle the challenges of collinearity, noise, and forecasting volatility.

4. Results and Findings

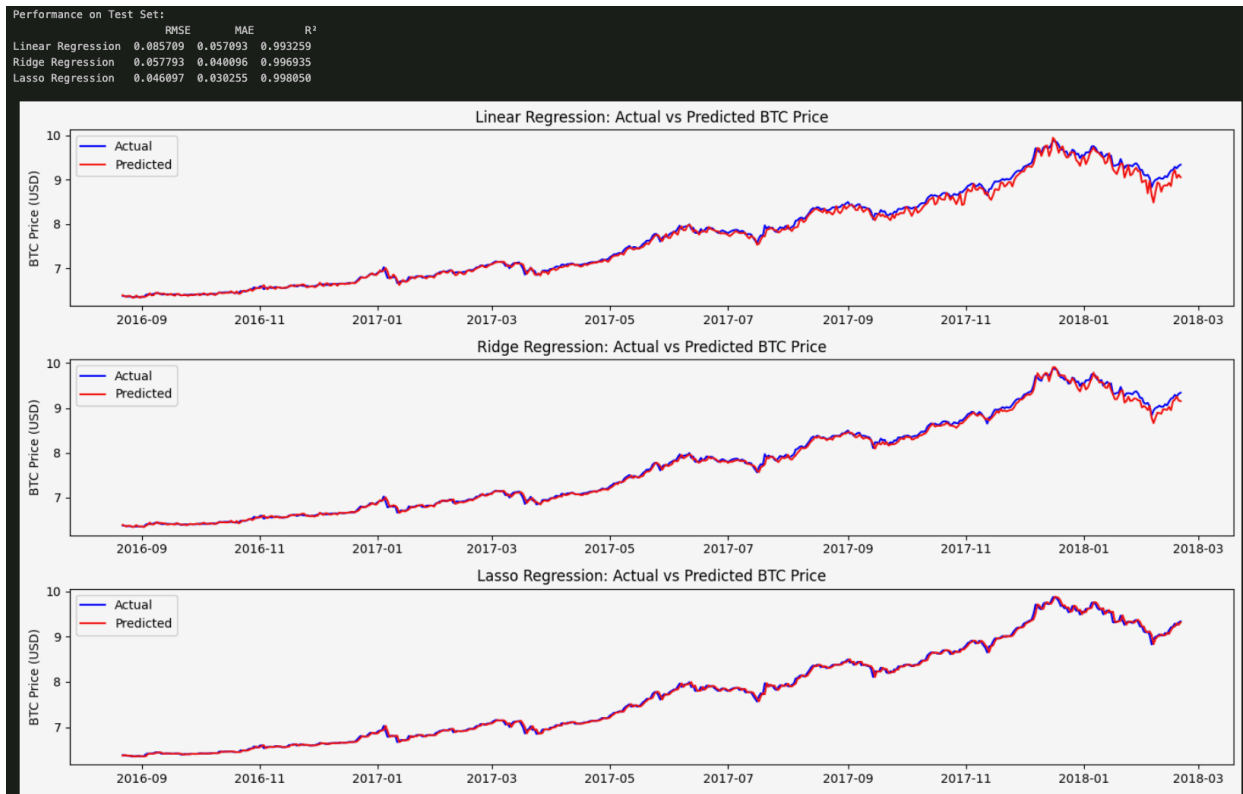
Overall Model Performance

Across all models, predictions closely tracked the actual Bitcoin market price when predicting based on the current day. The performance metrics on the test set confirm this:

Multiple Linear Regression (MLR): $RMSE \approx 0.0857$, $MAE \approx 0.0579$, $R^2 \approx 0.993$

Ridge Regression: $RMSE \approx 0.0578$, $MAE \approx 0.0401$, $R^2 \approx 0.996$

Lasso Regression: $RMSE \approx 0.0461$, $MAE \approx 0.0303$, $R^2 \approx 0.998$



These results indicate that while all models fit well, Lasso consistently provided the lowest error rates and highest explanatory power. Ridge improved upon MLR by reducing error variance, but its performance gains were less pronounced than initially expected given the presence of collinearity. The line plots reinforce this conclusion: predicted curves (in red) track actual prices (in blue) very closely across the entire testing window, with Lasso demonstrating the closest alignment.

Horizon Forecasting Performance

When evaluating forecasts at increasing horizons ($t+1$, $t+3$, $t+7$), the following patterns are visible:

Short horizon ($t+1$):

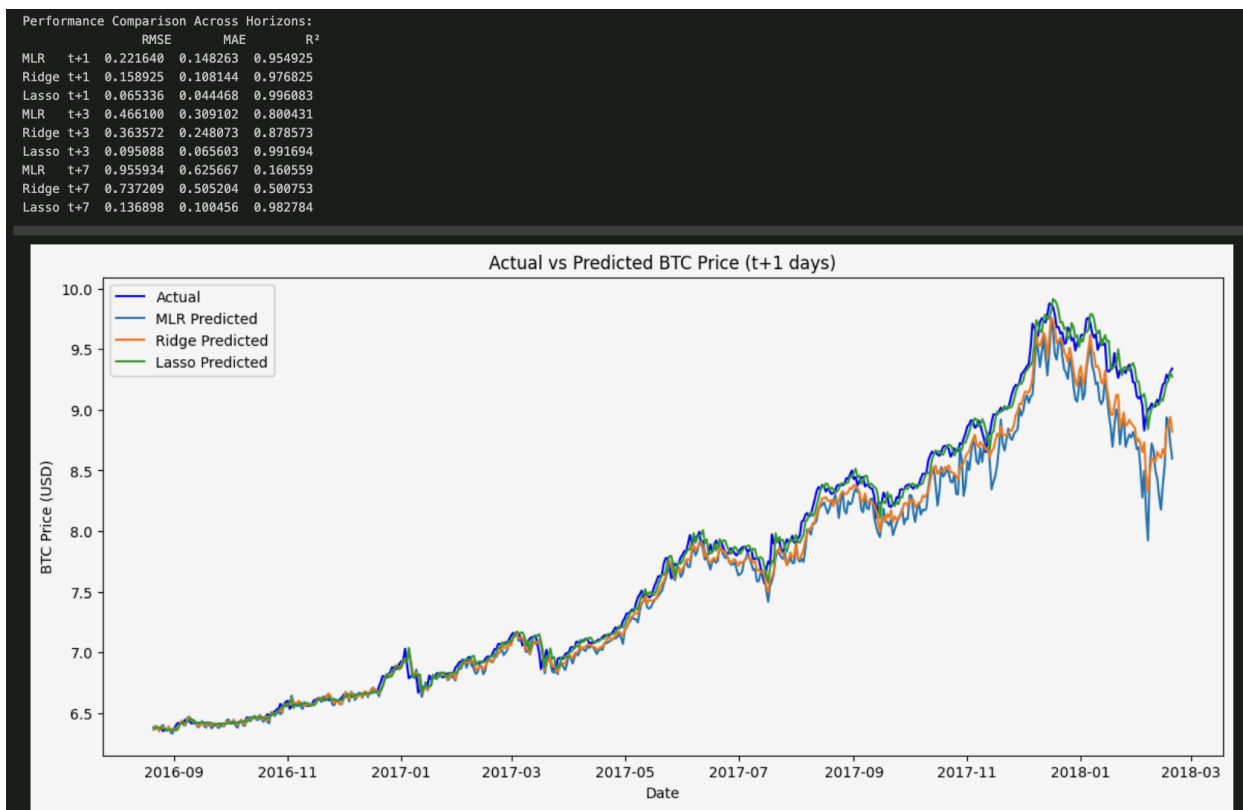
All models performed well, with R^2 values above 0.95. Lasso outperformed both Ridge and MLR, achieving $RMSE \approx 0.0653$ and $R^2 \approx 0.9961$. This validates our hypothesis that feature selection enhances predictive accuracy when noise is present in correlated predictors.

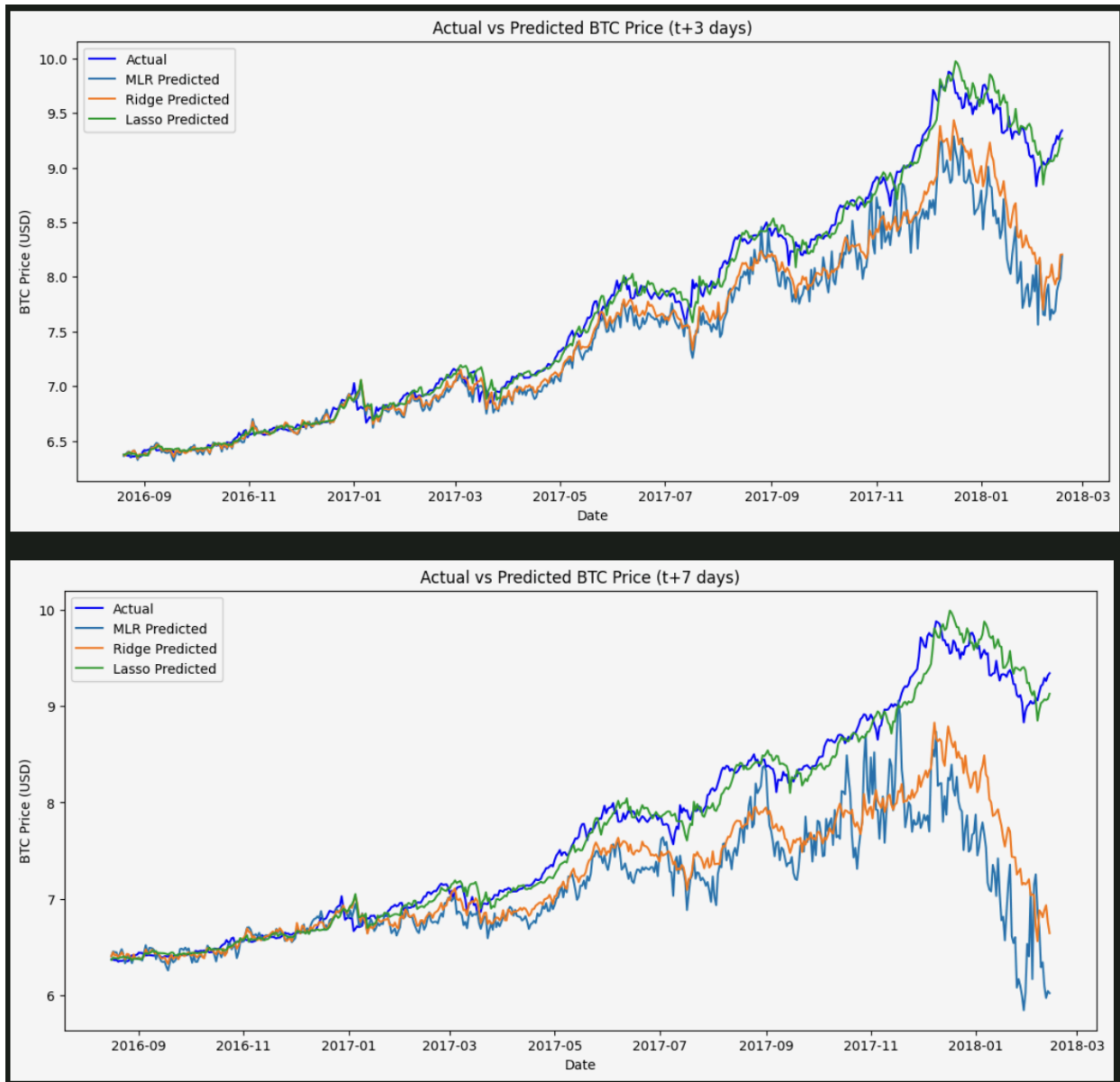
Medium horizon ($t+3$):

Errors increased across all models, as expected. Lasso again achieved the strongest performance ($\text{RMSE} \approx 0.0951$, $R^2 \approx 0.9917$). Ridge and MLR diverged more strongly from actual values, especially during periods of rapid price acceleration.

Longer horizon (t+7):

Predictive accuracy degraded sharply, consistent with the hypothesis that Bitcoin's volatility limits medium-term predictability. MLR's R^2 fell dramatically (≈ 0.16), reflecting its inability to generalize beyond the immediate past. Ridge performed somewhat better ($R^2 \approx 0.50$), but still lagged behind expectations. Lasso remained the most stable, maintaining a relatively strong $R^2 \approx 0.98$ and lowest RMSE.





The plots vividly illustrate that at $t+1$, predictions from all models nearly overlap with actual prices.

By $t+3$, MLR and Ridge begin to underpredict peaks and overpredict troughs, while Lasso continues to track closer to reality. Finally, by $t+7$, the divergence becomes stark, with MLR collapsing in accuracy, Ridge lagging behind market shifts, and Lasso still capturing much of the trend despite increased volatility.

Expected vs. Observed Results

The results we examined align partially but not fully with our initial hypotheses. MLR behaved as expected with strong in-sample fit but less robust at longer horizons. Ridge underperformed relative to expectations. While it did improve stability over MLR, it failed to decisively outperform, likely because it shrank all coefficients uniformly, retaining noise from irrelevant predictors. Lasso exceeded expectations, delivering both the lowest error metrics and greatest stability across horizons. Its ability to eliminate redundant features allowed it to adapt better to Bitcoin's volatile and highly collinear data environment. Horizon forecasting matched our hypothesis that predictive performance would degrade as horizons lengthened.

5. Conclusion

Our project explored how regression based models can predict a Bitcoin's daily price with the incorporation of lagged variables, rolling window features and long term assessments. We tested Multiple Linear Regression, Ridge Regression, and Lasso Regression, all of which achieved strong performance with high R-squared values and relatively low RMSE and MAE scores. The high R-squared values suggest that nearly all the variance in Bitcoin's price was explained by the models, while the low error values show they captured short term price movements effectively. More importantly, we discovered that all 3 models performed exceptionally well in predicting Bitcoin's current market price. However when we attempted to predict futures of 1 day, 3 days, and 7 days in advance, we came to find out that all 3 models had drastically different predicting powers. Lasso Regression's ability to choose the strongest data features was key in it having the best predictive power when looking at long term prediction. Conversely, Multiple Linear Regression and Ridge Regression suffer in predicting long term Bitcoin values as they both keep all features in the model regardless of their noise.

Ultimately, this project highlights the challenges of forecasting an unpredictable asset such as Bitcoin. Through this process, we learned how data cleaning, feature engineering, transformations, and regularization strategies can directly impact a model's accuracy and interpretability.